26 MIDDLEWARE vs GUARDS — WHY auth.guard.ts IS A GUARD, NOT MIDDLEWARE

Both middleware and guards run BEFORE the route handler, and both can reject a request outright. It's easy to think "auth.guard.ts checks a header and blocks bad requests — couldn't that just be middleware?" Technically yes; in practice Nest gives guards three things middleware doesn't, and this project's auth.guard.ts / roles.guard.ts rely on all three.

THE PIPELINE POSITION

```
Client Request
     |
MIDDLEWARE     <-- raw req/res, no Nest concepts yet
     |
GUARDS         <-- ExecutionContext, knows the target handler/class
     |
Interceptors (before)
     |
Pipes
     |
Route Handler
```

Middleware runs strictly earlier and knows nothing about which controller method is about to be called. Guards run just before the handler and are handed an ExecutionContext specifically because they need to reason about the target.

DIFFERENCE 1 — WHAT THEY'RE HANDED

Middleware:	use(req, res, next)	— raw Express objects only
Guard:	canActivate(context)	— ExecutionContext, which wraps req/res AND exposes context.getHandler() / context.getClass()

roles.guard.ts NEEDS that second part. It doesn't just look at the request — it asks "what @Roles(...) metadata is attached to the SPECIFIC handler being called?":

```
const requiredRoles = this.reflector.get<Role[]>(
  ROLES_KEY,
  context.getHandler(),
);
```

Middleware has no equivalent to context.getHandler(). It cannot see decorators/metadata on the controller method at all — by the time middleware runs, Nest hasn't even resolved which handler will be invoked yet in the way guards see it. This alone rules middleware out for anything metadata-driven (theory/23-reflector-and-metadata.txt).

DIFFERENCE 2 — HOW THEY REJECT A REQUEST

Middleware:	no built-in "rejected" concept. You manually do `res.status(401).json(...)` yourself and simply never call next(). You're back to hand-writing responses — exactly what theory/24-exception-handling.txt says this project never does.
Guard:	return false, or throw UnauthorizedException(...) / ForbiddenException(...). Nest's exception layer takes it from there and produces the same consistent JSON shape as every other throw in the app.

auth.guard.ts does exactly this:

```
if (key !== VALID_API_KEY) {
  throw new UnauthorizedException('Invalid or missing API key');
}
```

That single throw gets the same treatment as a service throwing NotFoundException — same filter, same response shape, zero manual res.json() anywhere. Middleware would have broken that consistency.

DIFFERENCE 3 — COMPOSING WITH ExecutionContext, NOT JUST ORDER

Both middleware and guards are injectable classes, so DI itself isn't the differentiator — it's that a guard can read the SAME ExecutionContext another guard/handler will see, letting them cooperate through the request object without either one hand-rolling its own req/res plumbing:

```
@Delete(':id')
@UseGuards(RolesGuard)        // AuthGuard already ran at controller
@Roles(Role.Admin)            // level (sets req.user in some setups);
remove(...) { ... }           // here RolesGuard resolves its own
                               // role from an x-role header directly
```

roles.guard.ts reads context.switchToHttp().getRequest(), attaches `request.user`, then reasons about it in the SAME canActivate call — exactly the object context.getHandler()/getClass() are describing. That symmetry (context, handler, class, request all reachable from one ExecutionContext) is what lets multiple guards stack meaningfully on one route. Middleware could technically mutate req the same way, but it would be reasoning blind about which handler is coming next.

SIDE BY SIDE

```
Middleware                          Guard
---------------------------------   ---------------------------------
Raw Express req/res, no context     ExecutionContext: handler+class+
                                     args, works across HTTP/WS/RPC
No metadata/reflection access       Reads @SetMetadata via Reflector
Manual res.status().json() to       throw HttpException -> goes
reject                              through the normal filter layer
Registered in configure() at the    Registered with @UseGuards() at
MODULE level                        method/controller/global level
Good for: logging, CORS, helmet,    Good for: authentication,
request IDs, raw header rewriting   authorization, anything reading
                                     route-specific metadata
```

★ RULE OF THUMB

If the check needs to know WHICH controller method is being called, or needs to read decorator metadata off it (@Roles, @Public, custom guards), it cannot be middleware — reach for a guard. If the check is identical for literally every request regardless of what handler it's headed to (logging, CORS, security headers), middleware is simpler and correctly placed earlier in the pipeline. auth.guard.ts and roles.guard.ts both need per-route metadata and Nest's exception handling, which is exactly why they were built as guards.

See also: **theory/25-middleware.txt**, **theory/17-guards.txt**, **theory/22-role-based-authorization.txt**, **theory/23-reflector-and-metadata.txt**, **theory/24-exception-handling.txt**